

Informe de Inspección Estática — Proyecto G8

Proyecto: Sistema de Gestión de Inventario (Estrucasa)

Stack: React 19 · Vite · Tailwind CSS v4 · react-router-dom v7

Persistencia: `localStorage` (sin backend)

Fecha de análisis: 2026-06-18

1. Componentización

Lo que funciona bien

- La separación `src/pages/` (wrappers de ruta) vs `components/<Feature>/` (lógica y UI) es una estructura razonable.
- `ItemProducto` e `ItemUsuario` encapsulan bien la fila de tabla, recibiendo callbacks (`onEdit`, `onDelete`, `onToggle`) desde el padre, lo que mantiene la lógica de mutación fuera de los hijos.
- `TablaEntradas` está correctamente extraída de `Entradas.jsx`, con sus propios filtros locales.
- `Dashboard.jsx` encapsula correctamente `getStorageArray` como helper local.

Problemas encontrados

Props sin desestructurar en componentes de fila

`ItemProducto` e `ItemUsuario` reciben el objeto de props completo con un nombre semánticamente confuso:

```
// ItemProducto.jsx:4
function ItemProducto(producto) {
  ...producto.stock... // no es el objeto producto, son las props de
  React
}

// ItemUsuario.jsx:4
function ItemUsuario(users) {
  ...users.nombre...
}
```

El parámetro se llama `producto` o `users`, pero en realidad es el objeto `props` de React. Además del nombre engañoso, este patrón hace que el linter de `react/prop-types` no pueda inferir los tipos correctamente.

Páginas que solo añaden un `<div>` vacío

Las páginas en `src/pages/` son wrappers que no aportan nada:

```
// entradasPage.jsx
function EntradasPage() {
```

```
    return (  
      <div>  
        <Entradas />  
      </div>  
    );  
  }  
}
```

La capa de página debería encargarse de cosas como el título de la pestaña (`<title>`), breadcrumbs, o lógica de carga por ruta. Sin esa responsabilidad, la capa es ruido.

Lógica de stock crítico duplicada en 5 lugares

La expresión `stock <= stockMinimo` para determinar si un producto es crítico aparece en:

Archivo	Línea
<code>Inventario.jsx</code>	88–90
<code>Entradas.jsx</code>	107–109
<code>Salidas.jsx</code>	75–79
<code>Dashboard.jsx</code>	39–43
<code>InventarioResumen.jsx</code>	32–37

Si la regla de negocio cambia (p. ej., crítico es `stock < stockMinimo`), hay que modificarla en 5 lugares distintos.

`getStorageArray` duplicado

La misma función auxiliar está copiada literalmente en `Dashboard.jsx` (línea 5) e `InventarioResumen.jsx` (línea 4). Debería vivir en un módulo compartido.

Datos mock duplicados entre módulos

Los tres usuarios seed están definidos dos veces: en `AuthContext.jsx` (`defaultUsers`, línea 21) y en `Usuarios.jsx` (`usuariosMock`, línea 6). Si cambia un dato de seed, hay que actualizarlo en ambos sitios.

2. Bugs y Problemas Funcionales

Bug crítico — Ediciones a usuarios seed siempre se revierten

`asegurarUsuariosPorDefecto` (llamada dentro de `login()`) sobrescribe los datos completos de los tres usuarios seed desde `defaultUsers`:

```
// AuthContext.jsx:81–90  
const mergedUser = {  
  ...currentUser,  
  id: defUser.id,
```

```
nombre: defUser.nombre,  
usuario: defUser.usuario,  
rol: defUser.rol,  
estado: defUser.estado,  
contrasena: defUser.contrasena, // sobrescribe contraseña guardada  
contraseña: defUser.contrasena  
};
```

Efecto: Un administrador puede editar a **jperez** en el módulo de usuarios (cambiarle rol, contraseña, o desactivarlo), y en el siguiente ciclo de login esos cambios se pierden y el usuario vuelve a **Administrador / 123456 / Activo**. Cualquier regla de negocio aplicada a los seeds es ignorada.

Bug alto — Ruta de imagen relativa rompe en build de producción

```
// Login.jsx:123  

```

Vite no procesa rutas relativas en atributos **src** de JSX a menos que se importen como módulos. En **npm run build**, el asset no se incluye en el bundle y la imagen aparecerá rota en producción. Debe importarse:

```
import logo from '../../../src/assets/LOGO.png';  
// y luego: <img src={logo} />
```

Bug medio — Keys de lista basadas en índice

En **Usuarios.jsx** (línea 197) e **Inventario.jsx** (línea 220):

```
key={"usuario" + index}  
key={"producto" + index}
```

Cuando el usuario filtra la lista, los índices cambian. React usará las keys equivocadas para reconciliar el DOM, pudiendo mantener el estado de un componente antiguo en uno nuevo. Las keys deberían ser los IDs únicos del registro (**key={usuario.id}**, **key={producto.id}**).

Bug medio — Enter en formularios de registro no guarda

RegistroUsuario.jsx y **RegistroProducto.jsx** tienen el botón guardar con **type="button"** y un **onClick**, pero el elemento **<form>** no tiene **onSubmit**:

```
// RegistroUsuario.jsx:216-219  
<button  
  type="button" // ← no dispara el submit del form
```

```
onClick={guardarUsuario}
>
```

Presionar Enter dentro de cualquier input no hace nada. La acción de guardar solo ocurre haciendo clic explícito.

Bug bajo — Campos de entrada omitidos en `TablaEntradas`

El formulario de Entradas captura `proveedor` y `observacion`, los guarda en `localStorage`, pero `TablaEntradas.jsx` solo muestra Producto, Cantidad, Responsable y Fecha. El proveedor es invisible en el historial.

Bug bajo — `InicioPage` navega a Dashboard sin verificar rol

```
// inicioPage.jsx:27
onClick={function() { navigate('/Dashboard'); }}
```

El rol `Operario` no tiene acceso a `/Dashboard`. Al hacer clic, `ProtectedRouteLayout` lo redirigirá inmediatamente a `/Entradas`. El botón funciona incorrectamente para Operarios.

Código muerto — `useFetch` nunca se usa

`src/hooks/useFetch.js` está definido pero no importado en ningún componente del proyecto. Es código muerto que puede eliminarse o documentarse si hay planes de migrar a una API real.

3. Gestión de Estado y Persistencia

Sin estado global compartido — datos desactualizados entre módulos

Cada módulo lee `localStorage` de forma aislada en su propio `useEffect` al montar. No hay contexto compartido para los datos de inventario, entradas o salidas.

Escenario de fallo: El usuario registra una entrada en `/Entradas`, aumentando el stock de un producto. Luego navega a `/Dashboard`. El Dashboard llama a `localStorage.getItem('inventario_app')` en su `useEffect` de montaje. Pero si el Dashboard ya estaba montado (React mantiene el árbol), el `useEffect` no vuelve a ejecutarse y los datos mostrados son los anteriores al registro.

Doble fuente de verdad y campo inconsistente

El campo contraseña tiene dos nombres distintos según el archivo:

Archivo	Campo
<code>AuthContext.jsx</code> (<code>defaultUsers</code>)	<code>contrasena</code>
<code>Usuarios.jsx</code> (<code>usuariosMock</code>)	<code>contraseña</code>
<code>RegistroUsuario.jsx</code>	<code>contraseña</code>

`AuthContext.jsx` lo compensa leyendo ambos:

```
const correctPassword = foundUser.contraseña || foundUser.contrasena;
```

Este parche funciona, pero es frágil: si un usuario se crea vía `RegistroUsuario` (guarda `contraseña`) y luego `asegurarUsuariosPorDefecto` lo procesa como seed (guarda `contrasena`), el campo `contraseña` sobrevive correctamente gracias al spread `...currentUser`, pero la inconsistencia es una trampa para futuros cambios.

Sesión no persistida entre recargas

`AuthContext` guarda el usuario en memoria (`useState`). Al recargar la página, la sesión se pierde y el usuario debe volver a autenticarse. No hay mecanismo de rehidratación desde `sessionStorage` o `localStorage`. Para un sistema de inventario operativo, esto limita la usabilidad.

IDs con `Date.now()` sin protección contra colisión

```
// RegistroProducto.jsx:102, RegistroUsuario.jsx:91  
id: Date.now()
```

Si dos registros se crean en el mismo milisegundo (posible con formularios con autoguardado), se generará un ID duplicado. Un UUID o un contador atómico sería más robusto.

4. Seguridad

Contraseñas en texto plano en `localStorage`

```
// localStorage['usuarios_app']  
[{ "usuario": "jperez", "contraseña": "123456" }, ...]
```

Cualquier script ejecutado en la página (incluyendo extensiones de navegador o XSS desde terceros) puede leer `localStorage` y obtener todas las credenciales. Para un proyecto universitario con datos ficticios el impacto es bajo, pero el patrón es incorrecto.

Control de acceso 100% en el cliente

Todo el sistema de permisos (`canAccessPath`, `rolePermissions`) está en JavaScript del lado del cliente. Abrir DevTools y ejecutar:

```
const u = JSON.parse(localStorage.getItem('usuarios_app'));  
u[2].rol = 'Administrador';  
localStorage.setItem('usuarios_app', JSON.stringify(u));
```

...le da a un Operario acceso completo al sistema tras recargar. No es un problema en ausencia de backend, pero debe documentarse como limitación de diseño.

Contraseñas hardcodeadas en el código fuente

```
// AuthContext.jsx:21-25
const defaultUsers = [
  { usuario: "jperez", contrasena: "123456" },
  ...
];
```

Estas credenciales quedan visibles en el bundle JS generado por Vite. Cualquiera que inspeccione el archivo `dist/assets/index-*.js` puede verlas.

5. Rendimiento

1800 partículas en la pantalla de login

```
// FondoParticulas.jsx:8
particleCount={1800}
```

Este valor es muy elevado. La librería `ogl` renderiza las partículas en WebGL, lo cual es eficiente, pero 1800 objetos con física en un canvas de pantalla completa puede causar caídas de framerate en dispositivos de gama media-baja. Un valor entre 200–400 produce el mismo efecto visual sin el costo.

`useMemo` ausente en el filtro de usuarios

`Salidas.jsx` y `Dashboard.jsx` usan `useMemo` para sus listas filtradas y cálculos derivados.

`Usuarios.jsx` no:

```
// Usuarios.jsx:95 – se recalcula en cada render
const usuariosFiltrados = usuarios.filter(function(usr) { ... });
```

Para listas pequeñas el impacto es mínimo, pero la inconsistencia es confusa.

`syncUser` ejecutada en cada cambio de ruta

```
// main.jsx:39-41
useEffect(() => {
  syncUser();
}, [location.pathname]);
```

Cada vez que el usuario navega, se llama a `syncUser`, que llama a `asegurarUsuariosPorDefecto`, que llama a `localStorage.getItem`, parsea el JSON, y potencialmente llama a `setUser`. En un árbol de componentes grande esto provoca renders innecesarios en cada navegación.

Sin paginación en tablas

Las tablas de Inventario, Entradas, Salidas y Usuarios renderizan todos los registros en el DOM de una sola vez. Con volúmenes grandes (cientos de entradas), el renderizado inicial se vuelve lento.

6. Convenciones de Código

Nomenclatura inconsistente

Problema	Ejemplo incorrecto	Corriente en el proyecto
<code>setState</code> sin camelCase	<code>setbusqueda</code> (<code>Usuarios.jsx:37</code>)	<code>setBusqueda</code> en <code>Inventario.jsx</code>
Rutas con mayúscula/minúscula mixtas	<code>/Inventarios</code> , <code>/Entradas</code>	<code>/usuarios</code>
Nombre de campo contraseña	<code>contrasena</code> en <code>AuthContext</code>	<code>contraseña</code> en <code>Usuarios</code> , <code>RegistroUsuario</code>

Estilo de función inconsistente

Algunos componentes usan arrow functions, otros usan declaraciones `function`:

```
// Login.jsx – arrow function
const Login = () => { ... }

// Inventario.jsx – function declaration
function Inventario() { ... }

// Sidebar.jsx – export default con function declaration
export default function Sidebar({ open, setOpen }) { ... }
```

No es incorrecto, pero la mezcla sin criterio dificulta la lectura.

`import React` innecesario

React 17+ (y React 19 aquí) no requiere importar React para usar JSX. Los siguientes archivos lo importan sin usarlo directamente:

- `ItemProducto.jsx` · `ItemUsuario.jsx` · `Inventario.jsx` · `Usuarios.jsx` · `Entradas.jsx` · `Login.jsx` · `TablaEntradas.jsx`

Comentarios que describen lo obvio

```
// ItemUsuario.jsx:37
{users.nombre.charAt(0).toUpperCase()} {/* Le pones en mayúscula al inicio
para sacar el avatar*/}

// ItemUsuario.jsx:40
<div className="font-semibold text-slate-800">{users.nombre}</div> {/*
Extraes el nombre del user */}
```

Los comentarios que parafrasean el código sin añadir contexto son ruido. Los comentarios útiles explican el *porqué*, no el *qué*.

Botones sin **type** explícito dentro de formularios

En **Inventario.jsx**, el botón "Agregar Producto" dentro del contenedor del filtro no tiene **type="button"**. Si ese contenedor alguna vez se envuelve en un **<form>**, el botón disparará un submit no deseado.

7. Resumen por Severidad

CRÍTICO

#	Descripción	Archivo	Impacto
C1	Ediciones a usuarios seed se revierten en cada login	AuthContext.jsx:81–90	Funcional total: los cambios a jperez/atorres/lramos no persisten
C2	Imagen de logo con ruta relativa — rompe en producción	Login.jsx:123	Visual: imagen rota en npm run build

ALTO

#	Descripción	Archivo	Impacto
A1	Sin estado compartido: Dashboard y Reportes muestran datos desactualizados	main.jsx , todos los módulos	Funcional: datos incorrectos sin recargar
A2	Contraseñas en texto plano en localStorage	AuthContext.jsx , Usuarios.jsx	Seguridad: credenciales expuestas
A3	Doble definición de usuarios seed con campo contrasena/contraseña inconsistente	AuthContext.jsx , Usuarios.jsx	Mantenibilidad: bug latente

MEDIO

#	Descripción	Archivo	Impacto
M1	Keys de lista basadas en índice	<code>Usuarios.jsx:197</code> , <code>Inventario.jsx:220</code>	Bugs de renderizado con filtros activos
M2	Lógica de stock crítico duplicada en 5 archivos	Múltiples	Mantenibilidad: cambio de regla requiere 5 ediciones
M3	Botón guardar con <code>type="button"</code> en formularios	<code>RegistroUsuario.jsx:216</code> , <code>RegistroProducto.jsx:239</code>	UX: Enter no guarda
M4	Control de acceso 100% en el cliente, alterable vía DevTools	<code>AuthContext.jsx</code>	Seguridad: limitación de diseño
M5	1800 partículas en login	<code>FondoParticulas.jsx:8</code>	Rendimiento en dispositivos lentos
M6	<code>useFetch</code> definido pero nunca utilizado	<code>src/hooks/useFetch.js</code>	Código muerto / confusión

BAJO

#	Descripción	Archivo	Impacto
B1	<code>setbusqueda</code> sin camelCase	<code>Usuarios.jsx:37</code>	Convención
B2	<code>import React</code> innecesario en 7 archivos	Múltiples	Convención
B3	Props sin desestructurar en <code>ItemProducto</code> e <code>ItemUsuario</code>	<code>ItemProducto.jsx:4</code> , <code>ItemUsuario.jsx:4</code>	Legibilidad
B4	<code>getStorageArray</code> duplicado	<code>Dashboard.jsx:5</code> , <code>InventarioResumen.jsx:4</code>	Mantenibilidad
B5	Botón "Ir al Dashboard" inaccesible para Operarios	<code>inicioPage.jsx:27</code>	UX menor
B6	Proveedor y observación guardados pero no mostrados en historial	<code>TablaEntradas.jsx</code>	Funcional menor
B7	Sesión no persistida: logout al recargar	<code>AuthContext.jsx</code>	UX
B8	Sin paginación en tablas	Todos los módulos	Rendimiento con volumen
B9	Comentarios que describen el código en lugar de el porqué	<code>ItemUsuario.jsx</code> , otros	Convención

8. Recomendaciones Prioritarias

Las siguientes acciones tienen el mayor retorno en relación al esfuerzo:

1. Corregir el seed override (C1) — estimado: 30 min

En `asegurarUsuariosPorDefecto`, reemplazar la lógica de merge para que solo inserte un seed si no existe, sin sobrescribir datos existentes:

```
if (existingIndex === -1) {  
  users.push(defUser); // solo inserta si no existe  
  modificado = true;  
}  
// eliminar el bloque de merge/update completo
```

2. Corregir la ruta del logo (C2) — estimado: 5 min

```
// En Login.jsx, agregar al tope:  
import logo from '../src/assets/LOGO.png';  
// y usar: <img src={logo} />
```

3. Centralizar la lógica de stock crítico (M2) — estimado: 20 min

Crear `src/utils/inventario.js` con funciones `esCritico(producto)` e `isStockEnRiesgo(producto)` e importarlas en los 5 módulos.

4. Unificar el nombre del campo contraseña (A3) — estimado: 20 min

Elegir `contrasena` (sin tilde, más seguro en JSON cross-platform) y estandarizarlo en todos los archivos. Eliminar el fallback `foundUser.contrasena || foundUser.contrasena`.

5. Corregir keys de lista (M1) — estimado: 5 min

Reemplazar `key={"usuario" + index}` por `key={usuario.id}` en `Usuarios.jsx` y equivalente en `Inventario.jsx`.

6. Reducir partículas (M5) — estimado: 2 min

En `FondoParticulas.jsx`, cambiar `particleCount={1800}` a `particleCount={300}`.

7. Añadir `onSubmit` a formularios de registro (M3) — estimado: 10 min

En `RegistroUsuario.jsx` y `RegistroProducto.jsx`, añadir `onSubmit={guardarUsuario}` al `<form>` y cambiar el botón a `type="submit"`.

8. Eliminar `useFetch` o documentarlo (M6) — estimado: 5 min

Si no hay planes de conectar a una API, borrar el archivo. Si los hay, documentarlo con un comentario que indique para qué endpoint está pensado.